

U.S. SEMICONDUCTOR: TRADE & LABOR ANALYSIS

Logistics Risk — Rail Equipment Incident Data (U.S. DOT Form 54) | 2024–2025

Overview

This document describes the data cleaning steps applied to the U.S. DOT Rail Equipment Accident/Incident dataset (Form 54), covering incidents reported between January 1, 2024 and January 1, 2026. The objective was to simplify the dataset so that it focuses on variables relevant to railroad income, logistics resilience, and business impact — removing redundant identifiers, technical codes, and metadata that added no analytical value, while retaining financial, operational, and organizational fields.

The cleaned output is saved as `Clean_Rail_Equipment_Accident_Incident_Data_(Form_54)_20260121.csv` and is ready for risk, resilience, and supply chain analysis.

Dataset Summary

Attribute	Value
Source file	<code>Rail_Equipment_Accident_Incident_Data_(Form_54)_20260121.csv</code>
Output file	<code>Clean_Rail_Equipment_Accident_Incident_Data_(Form_54)_20260121.csv</code>
Time window applied	January 1, 2024 — January 1, 2026
Reporting form	U.S. DOT Form 54 — Rail Equipment Accident/Incident Report
Data source	<code>data.transportation.gov</code>
Analytical focus	Damage costs, train logistics, hazmat incidents, evacuation counts, railroad business identifiers

Step 1 — Reading the Raw Dataset

Objective

Load the original accident/incident dataset into a pandas DataFrame for inspection and transformation.

Code

```
df = pd.read_csv(
    'Rail_Equipment_Accident_Incident_Data_(Form_54)_20260121.csv'
)
```

Key Design Choice

- **Raw file scope:** The file contained many fields including multiple accident identifiers, technical codes, and metadata that were not useful for business or logistics analysis. Loading into pandas provided full visibility before any transformation.

Step 2 — Converting the Date Column

Objective

Convert the Date column into proper datetime format to enable accurate time-based filtering and trend analysis.

Code

```
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
```

Key Design Choice

- **errors='coerce':** Ensures that invalid or malformed date entries are safely converted to NaT (Not a Time) rather than raising an exception, avoiding calculation errors in time-based filtering.

Step 3 — Filtering by Time Window

Objective

Restrict the dataset to the most relevant period for current analysis — January 2024 through January 2026 — reducing dataset size and ensuring analytical focus.

Code

```
df = df[(df['Date'] >= '1/1/2024') & (df['Date'] <= '1/1/2026')]
df = df.reset_index(drop=True)
```

Key Design Choice

- **Time window:** Narrowing to a two-year window captures the most operationally relevant incidents for current logistics and supply chain analysis while excluding historical data that may reflect outdated infrastructure or reporting practices.
- **reset_index:** Resets the DataFrame index after filtering to ensure clean, sequential row numbering for downstream processing.

Step 4 — Dropping Redundant or Misleading Columns

Objective

Remove columns that add no analytical value, are redundant with retained fields, or introduce unnecessary complexity for business and logistics analysis.

Code

```
cols_to_drop = [
    'Accident Number', 'Other Accident Number', 'Maintenance Accident Number',
    'Division', 'Division Code', 'Other Railroad Code', 'Accident Type Code',
    'Visibility Code', 'Weather Condition Code', 'Track Type Code',
    'Train Direction Code', 'Equipment Type Code', 'Signalization Code',
    'Method of Operation Code', 'Adjunct Code 1', 'Adjunct Code 2', 'Adjunct Code
3',
    'Remote Control Locomotive Code', 'First Car Initials', 'First Car Number',
    'First Car Position', 'Causing Car Initials', 'Causing Car Number',
    'Causing Car Position', 'Hours Engineers On Duty', 'Minutes Engineers On Duty',
    'Hours Conductors On Duty', 'Minutes Conductors On Duty', 'PDF Link',
    'Incident Key', 'Report Key', 'Special Study 1', 'Special Study 2',
    'County Code', 'State Code', 'Temperature', 'Visibility',
    'Weather Condition', 'Other Accident Month', 'Grade Crossing ID'
]
df = df.drop(columns=cols_to_drop)
```

Rationale for Each Dropped Column Group

Column Group	Reason for Removal
Accident Number, Other Accident Number, Maintenance Accident Number	Multiple redundant identifiers — no additional analytical value
Division, Division Code, Other Railroad Code	Internal organizational codes not needed for business-level analysis
Accident Type Code, Track Type Code, Equipment Type Code, Signalization Code, Method of Operation Code	Technical operational codes — not central to financial or logistics focus
Visibility Code, Weather Condition Code, Temperature, Visibility, Weather Condition	Environmental metadata not relevant to income/logistics/business analysis
Adjunct Code 1/2/3, Remote Control Locomotive Code	Operational detail codes adding no value to business impact analysis
First Car Initials/Number/Position, Causing Car Initials/Number/Position	Car-level granularity not needed for aggregate logistics analysis
Hours/Minutes Engineers On Duty, Hours/Minutes Conductors On Duty	Crew duty minutiae too granular for financial/logistics focus
PDF Link, Incident Key, Report Key, Special Study 1/2	Metadata and internal system fields — not analytical
County Code, State Code	Duplicated by full county/state name fields retained in dataset
Other Accident Month, Grade Crossing ID	Operational fields not relevant to the business analysis scope

Step 5 — Handling Missing Values

Objective

Standardize missing values across the dataset so they are consistently represented and correctly interpreted by analytical tools.

Code

```
# Step 1: Replace blanks with string placeholder for visibility
df = df.fillna('NaN')

# Step 2: Convert string placeholder back to proper np.nan
df.replace('NaN', np.nan, inplace=True)
```

Key Design Choice

- **Two-step NaN handling:** Filling with 'NaN' first ensures that blank cells are made visible and consistent. Converting back to np.nan then ensures that missing values are correctly recognized as null by pandas and all downstream analytical tools — avoiding confusion between empty strings and true missing data.

Step 6 — Saving the Cleaned Dataset

Objective

Export the cleaned dataset as a structured CSV file ready for risk analysis, logistics modeling, and supply chain resilience assessment.

Code

```
df.to_csv(
    'Clean_Rail_Equipment_Accident_Incident_Data_(Form_54)_20260121.csv',
    index=False
)
```

Fields Retained in the Cleaned Dataset

Field Category	Examples of Retained Fields
Financial / Damage	Total damage cost, equipment damage, track damage
Logistics / Operations	Tons of freight, number of cars derailed, number of locomotives
Hazmat & Safety	Hazmat released, evacuation counts
Railroad Identifiers	Railroad name, reporting railroad
Geography	State, city, county (full names)
Temporal	Date (datetime), month, year

Key Decisions and Rationale

- **Accident identifiers dropped:** Multiple accident numbers and internal codes were redundant and added no value to business or logistics analysis.
- **Weather and visibility removed:** These fields were not central to income, logistics, or business analysis and risked distracting from core financial and operational metrics.
- **Crew duty minutiae excluded:** Hours and minutes on duty were too granular and not relevant to the financial and logistics focus of this analysis.
- **Business focus sharpened:** By retaining railroad identifiers, damage costs, tonnage, and car counts, the dataset directly supports analysis of operational resilience and business impact.
- **Two-year time window:** Restricting to 2024–2026 ensures relevance to current supply chain conditions while keeping the dataset manageable.
- **Transparency ensured:** Every cleaning step is documented, making the dataset fully reproducible and audit-ready.

Data Source

[U.S. Department of Transportation — Rail Equipment Accident/Incident Data \(Form 54\)](#)